Dynamic Programming Study Guide

Practice Session on Dynamic Programming

Catalan Numbers

What are Catalan Numbers?

Catalan numbers are a sequence of natural numbers that occur in various counting problems. The nth Catalan number is given by the formula:

$C(n) = (2n)! / ((n+1)! * n!)$

The sequence starts: 1, 1, 2, 5, 14, 42, 132, 429, ...

Dynamic Programming Approach for Catalan Numbers

The Catalan numbers follow the recurrence relation:

- $C(0) = 1$
- $C(n) = \Sigma(i=0 \text{ to } n-1)\ C(i) * C(n-1-i)$

```java
public class CatalanNumbers {
    // Method to find nth Catalan number using Dynamic Programming
    public static long catalanDP(int n) {
        if (n <= 1) return 1;

        // Create array to store results of subproblems
        long[] catalan = new long[n + 1];

        // Initialize first two values
        catalan[0] = 1;
        catalan[1] = 1;

        // Fill entries using the recurrence relation
        for (int i = 2; i <= n; i++) {
            catalan[i] = 0;
            for (int j = 0; j < i; j++) {
                catalan[i] += catalan[j] * catalan[i - 1 - j];
            }
        }

        return catalan[n];
    }

    public static void main(String[] args) {
        for (int i = 0; i < 10; i++) {
            System.out.println("C(" + i + ") = " + catalanDP(i));
        }
    }
}
```

Applications of Catalan Numbers

1. **Number of ways to parenthesize expressions**
2. **Number of binary search trees with n nodes**
3. **Number of paths in a grid that don't cross the diagonal**
4. **Number of ways to triangulate a polygon**

Mixed Dynamic Programming Questions

1. Fibonacci Sequence with Memoization

```java
import java.util.HashMap;
import java.util.Map;

public class FibonacciDP {
    private static Map<Integer, Long> memo = new HashMap<>();

    public static long fibonacci(int n) {
        if (n <= 1) return n;

        if (memo.containsKey(n)) {
            return memo.get(n);
        }

        long result = fibonacci(n - 1) + fibonacci(n - 2);
        memo.put(n, result);
        return result;
    }

    // Iterative approach
    public static long fibonacciIterative(int n) {
        if (n <= 1) return n;

        long[] dp = new long[n + 1];
        dp[0] = 0;
        dp[1] = 1;

        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }

        return dp[n];
    }
}
```

2. Climbing Stairs Problem

You're climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

```java
public class ClimbingStairs {
    public static int climbStairs(int n) {
        if (n <= 2) return n;

        int[] dp = new int[n + 1];
        dp[1] = 1;
        dp[2] = 2;

        for (int i = 3; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }

        return dp[n];
    }

    // Space optimized version
    public static int climbStairsOptimized(int n) {
        if (n <= 2) return n;

        int prev2 = 1, prev1 = 2;

        for (int i = 3; i <= n; i++) {
            int current = prev1 + prev2;
            prev2 = prev1;
            prev1 = current;
        }

        return prev1;
    }
}
```

3. House Robber Problem

You are a robber planning to rob houses along a street. Each house has a certain amount of money. Adjacent houses have security systems connected, so you cannot rob two adjacent houses.

```java
public class HouseRobber {
    public static int rob(int[] nums) {
        if (nums.length == 0) return 0;
        if (nums.length == 1) return nums[0];

        int[] dp = new int[nums.length];
        dp[0] = nums[0];
        dp[1] = Math.max(nums[0], nums[1]);

        for (int i = 2; i < nums.length; i++) {
            dp[i] = Math.max(dp[i - 1], dp[i - 2] + nums[i]);
        }

        return dp[nums.length - 1];
    }

    // Space optimized version
    public static int robOptimized(int[] nums) {
        if (nums.length == 0) return 0;
        if (nums.length == 1) return nums[0];

        int prev2 = nums[0];
        int prev1 = Math.max(nums[0], nums[1]);

        for (int i = 2; i < nums.length; i++) {
            int current = Math.max(prev1, prev2 + nums[i]);
            prev2 = prev1;
            prev1 = current;
        }

        return prev1;
    }
}
```

4. Unique Paths in Grid

A robot is located at the top-left corner of an m x n grid. The robot can only move either down or right. How many possible unique paths are there?

```java
public class UniquePaths {
    public static int uniquePaths(int m, int n) {
        int[][] dp = new int[m][n];

        // Initialize first row and column
        for (int i = 0; i < m; i++) {
            dp[i][0] = 1;
        }
        for (int j = 0; j < n; j++) {
            dp[0][j] = 1;
        }

        // Fill the dp table
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
            }
        }

        return dp[m - 1][n - 1];
    }

    // Space optimized version
    public static int uniquePathsOptimized(int m, int n) {
        int[] dp = new int[n];

        // Initialize with 1s
        for (int j = 0; j < n; j++) {
            dp[j] = 1;
        }

        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[j] = dp[j] + dp[j - 1];
            }
        }

        return dp[n - 1];
    }
}
```

5. Longest Common Subsequence (LCS)

Find the length of the longest common subsequence between two strings.

```java
public class LongestCommonSubsequence {
    public static int longestCommonSubsequence(String text1, String text2) {
        int m = text1.length();
        int n = text2.length();

        int[][] dp = new int[m + 1][n + 1];

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }

        return dp[m][n];
    }
}
```

Quiz Questions

Question 1: What is the 4th Catalan number (C(4))? a) 5 b) 14 c) 42 d) 132

Answer: b) 14

Question 2: In the climbing stairs problem, if there are 5 stairs, how many distinct ways are there to climb to the top? a) 5 b) 8 c) 13 d) 21

Answer: b) 8

Question 3: What is the time complexity of the dynamic programming solution for finding the nth Fibonacci number? a) $O(n^2)$ b) O(n) c) O(log n) d) $O(2^n)$

Answer: b) O(n)

Question 4: In the House Robber problem, what is the main principle behind the dynamic programming approach? a) Always rob the house with maximum money b) For each house, choose the maximum between robbing current house + best from two houses back, or best from previous house c) Rob every alternate house d) Rob houses in random order

Answer: b) For each house, choose the maximum between robbing current house + best from two houses back, or best from previous house

Question 5: What is the space complexity of the optimized version of the Unique Paths problem? a) O(m × n) b) O(m + n) c) O(min(m, n)) d) O(1)

Answer: c) O(min(m, n))

Practice Coding Questions

Question 1: Write a Java program to find the minimum number of coins needed to make a given amount using coins of denominations [1, 5, 10, 25]. Use dynamic programming approach.

Question 2: Implement a solution for the "Maximum Subarray Sum" problem using Kadane's algorithm with dynamic programming. Given an array of integers, find the contiguous subarray with the largest sum.

Question 3: Create a Java program to solve the "Edit Distance" problem. Given two strings, find the minimum number of operations (insert, delete, replace) required to convert one string into another using dynamic programming.

Question 4: Write a program to find the number of ways to decode a string of digits where 'A' = 1, 'B' = 2, ..., 'Z' = 26. For example, "226" can be decoded as "BZ" (2,26), "VF" (22,6), or "BBF" (2,2,6).

Question 5: Implement the "Palindromic Subsequence" problem: Given a string, find the length of the longest palindromic subsequence using dynamic programming approach.